

Backend Developer Challenge

Objective

Build a backend application in Python/Django that can:

1. Read a candidate's resume (uploaded as a file, such as a PDF).
2. Compare the resume with a list of job postings.
3. Provide a detailed output that includes:
 - a. **The match percentage between the candidate and each job.**
 - b. **A breakdown of what aligns with the job requirements (skills, experience, etc.).**
 - c. **A summary of gaps where the candidate's profile doesn't meet the job's criteria.**

What We're Looking For

Your application should:

1. **Handle Inputs:**
 - a. Allow uploading a candidate's resume (file format).
 - b. Fetch job postings via an API or endpoint.
2. **Process Data:**
 - a. Extract relevant details from the resume (e.g., skills, experience, education, etc).
 - b. Match the extracted data against the job requirements (like skills, languages, etc).
 - c. Calculate a match percentage using a basic AI service or algorithm.
3. **Provide Outputs:**
 - a. **Display a ranked list of job postings based on the match percentage.**
 - b. **Include a detailed breakdown of:**
 - i. **What aspects of the candidate align with the job.**
 - ii. **Any areas where the candidate's profile falls short.**

Technical Expectations

- Use **Django** to develop the backend.
- Expose the results via a **GraphQL API** (or REST API, if preferred).

- Leverage the **Django ORM** for handling database operations.
- Provide a **Dockerfile** to containerize the application with clear instructions to run it.
- Include a **Jenkinsfile** to outline a basic CI/CD pipeline (no need to execute it).

Bonus Points

- Use **Delta Table/Delta Lake** to store job data or match results.
- Implement a basic **AI service** for natural language processing. For instance:
 - Use open-source tools like **spaCy** or **NLTK** to extract and compare keywords.
- Add a simple **Next.js interface** (optional):
 - A form to upload resumes.
 - A page to view the match results in an easy-to-read format.

What to Deliver

1. A Git repository with:
 - a. The application code.
 - b. A clear README with instructions to set up and run the project.
 - c. Docker and Jenkins configurations.
2. Example resumes and job postings to test the functionality.
3. (Optional) A short video or document explaining your approach and key decisions.

How We'll Evaluate

1. **Functionality:**
 - a. Does the app correctly match resumes with job postings?
 - b. Is the output clear, accurate, and actionable?
2. **Code Quality:**
 - a. Is the code well-structured, clean, and easy to understand?
 - b. Are Django ORM queries used effectively?
3. **Technical Skills:**
 - a. Is Docker set up correctly?
 - b. Does the CI/CD pipeline make sense, even if not executed?
4. **Extra Mile:**
 - a. Using GraphQL instead of REST.

- b. Incorporating Delta Table/Delta Lake.
- c. Implementing AI-based keyword matching.
- d. Adding a polished Next.js interface.

Timeline

- You'll have **5-7 days** to complete the challenge.

If you have any questions or need clarification, feel free to ask –
rani.souza@wearemeta.io

Good luck, and we're excited to see what you build! 😊